



Helping Hands

Development Team Handbook

1 Introduction

Welcome to the Helping Hands Development Team handbook. This handbook is intended as a guideline document introducing new and existing staff to the functions, responsibilities, and expectations of the Development Team.

Our community is designed to bridge Deaf and hearing communities through the use of technology. It is thus important that we make technology work for us and the community.

Your area of technical expertise, as well as your level of proficiency in that area, will vary between you and your fellow team members. This is OK, and where possible you should see it as an opportunity to both contribute in ways others may not be able to, and learn and improve in areas you may not have otherwise expected to.

If you run into problems or are uncertain of how to proceed, you are always welcome to seek guidance within the #dev-team-chat.

By being selected to be part of this team, it means that you have received a certain amount of trust, and that we believe you to both understand and be able to make practical contributions towards the goals of Helping Hands. As a result, it is expected you may already inherently understand many of the principles we work by, and the expectations laid upon you.

This document primarily serves to codify many of the principles typically inferred or left unspoken, and you are free to scan through the document and return to any sections of interest at a later time if necessary. You may not need to read or understand the full content to do your work effectively. However, if you do find any sections or elements confusing, please make these concerns known as it can help refine the document for future readers.

Thank you for taking the time to read this handbook and for being part of our team.

2 Philosophy of Development

This section covers abstract concepts that relate to your development work at Helping Hands. Fine implementation details and the more pragmatic aspects of design will be covered in other sections – the goal of this section is centered more around the ethical principles you should apply in the development of your projects.

• Access

One of the core issues encountered by Deaf people in interactions with society is the issue of access. While lack of accommodations for hearing loss or difficulty often manifests in human factors, barriers to access also result from implementations of technology.

In general, it is expected that you will design your projects to pay due attention to the issues of access. Our community is diverse, and their feedback will be one of the most valuable resources in achieving these goals.

It is important to realize that access does not just mean access for the Deaf community. There are many other forms of limitations to access that we can inadvertently cause, and we should not let our focus on Deaf issues, hearing loss, and sign language allow us to neglect other access requirements, such as motor conditions, sight loss, sensory impairment, and language disabilities.

While it may not be possible to always engineer a system that works for every person, considering a broad range of needs that the people affected by your technology may have is important in ensuring that we serve as an unbiased representative of access.

- **Societal Impact**

While Helping Hands is, in the grand scope of things, a small community in a niche subject area, it is perhaps because we are unique in our positioning that we have to exercise caution about the decisions we make.

There exist communities, including Deaf communities, that have reason to be skeptical about the positioning of Helping Hands, and may be apprehensive about whether we represent a positive effect on the image of the Deaf community and sign language.

Indeed, for the majority of our users, Helping Hands may be the first or only time they interact with Deaf people and the Deaf community as a whole. We must be careful that we impart respectful and reasonable values, and that we do not inadvertently contribute to misconceptions or the erasure of Deaf culture outside of the walls of our community.

There will always be some amount of skepticism around new technology and the place it takes within people's lives, but by being careful in our execution we hope to set a rare example of how Helping Hands helps to contribute to the world more than it detracts from it.

- **Commercial Positioning**

Helping Hands is comprised entirely of volunteers, such as yourself. We are not a business, and we are also not a registered charity. The position this puts us in leaves us with implications from both a legal and ethical standpoint.

Where possible, we should decouple any form of commercial usage or goals from the Helping Hands community. This means that we should never handle money internally, and we should not employ monetization or financial systems in the projects we develop under our name.

Your participation in Helping Hands does not prevent you from developing your own work for commercial purposes. However, such work should not be under the Helping Hands name. Helping Hands is about access and goodwill, and there should never be any monetary incentives or profit involved in the work we directly perform under our team.

- **Security & Data Privacy**

Responsible handling of user data is a pertinent issue in all forms of software development. We are fortunate in that, for the most part, we are able to avoid handling end user data directly, but there may be times where doing so is desirable or necessary.

You are obliged to comply with data protection legislation employed in both your own region and the regions of your users. As many software pipelines involve passing or processing data in some capacity between different global entities, you should comply at a minimum with GDPR and CCPA requirements.

You should also ensure that any machines, software, or accounts you use to collect or process data from our userbase is suitably secure. If you integrate with SaaS solutions, pay for hardware or network hosting, or use data platforms (including ones that perform surveys, telemetry, analytics, or profiling), you should check what data policies they have declared compliance with.

Be careful about credential management when it is relevant - do not store passwords, API keys, tokens or similar in source code, or in locations where they may be easily exfiltrated.

You are also encouraged to rely on known-safe and known-secure implementations when it comes to handling of authentication and other software areas with nuanced security characteristics. There are many useful sources of info on up to date security practice, such as OWASP's Cryptographic Storage Cheat Sheet [1].

The best approach to sensible data handling is always to avoid handling data if you can, and avoid storing or sharing it if you can't.

- **Legal and Contractual Compliance**

In general, much of our development work integrates with existing platforms and technology. In addition to the baseline legal requirements associated with any development work, you must also consider your work's compliance with usage policies, including your rights and any limitations that may be placed on you.

In many cases, tools or platforms you use will present you with a **license agreement**. This is a document that requires you to agree to comply with a set of rules at a legal capacity in order to receive a **license**, a document which permits you to certain usage freedoms you would not have inherent legal right to.

While reading and understanding every possible license agreement you may be faced with is usually not possible, you should try to ensure that you understand any limitations and obligations placed upon you for the specific forms of integration you are implementing.

As Helping Hands is not a legal entity, it cannot take legal liability for you. You have a responsibility to ensure on your own that you are in compliance, but your team can assist you with double checking if you are unsure.

These are just some of the areas you are expected to consider in your work. In most situations, solving and accounting for these issues does not need to be complicated, and through cooperation with your team and the community you will likely gain an intuitive sense of how to approach these in your development workflow.

③ Project Lifecycle

This section is dedicated to the discussion of managing your own projects for the benefit of yourself and the team.

For most developers, but especially those that are newer to the field, the process of predicting and planning for the lifecycle your project will take can be incredibly difficult. It is very easy to get carried away with the excitement of working on something new, without considering how the project will be handled and used in practice months or years down the line.

It is especially important to consider the position that you, as a lead developer of the project, represent in the project's long term outcome. In much the same way that we expect to take on and introduce new developers to the team over time, we also expect that developers will most likely offboard and become unable or unwilling to directly maintain the work they have done for us eventually.

As time goes on, the circumstances of your life and what you consider your priorities may change. It is not generally possible to predict when these things happen, and life may not give you the opportunity to hand off your work gracefully. It is thus important that your considerations for how the project will be maintained are considered during the course of your work.

The form that project lifecycle management takes is likely to vary from project to project. However, most work that is related to or can be considered to be “software” is best managed with the usage of version control solutions.

Helping Hands has a [GitHub organization](#)  that you can request access to and maintain either public or private repositories within. This service allows you to version control source code and assets with the [Git Version Control System](#) . It is the recommended way to handle direct software, as well as software-like projects such as Unity projects.

You are generally entrusted with the agency to work on whatever projects you think will benefit the community, and you usually do not need permission to start working on something as long as it aligns with the principles mentioned in Section 2. You should still communicate what are you working on, so that others can assist you and ensure they are not repeating your work.

Whether you choose to make your source code and project available to the wider public is a personal decision on your part as a developer. This includes what kind of license you grant when doing this publication, although as an access-focused community we highly recommend OSI-approved licenses without restrictive clauses, as to allow community members to make the best use of the technology we have developed.

When you produce work for Helping Hands, you for all intents and purposes retain your ownership and licensing rights over that work. However, we request and expect that you grant Helping Hands the license to modify, maintain, copy, derive, share, relicense, and redistribute the work you have produced. If possible, we do our best to explicitly request the permission to do any of these as they become relevant, and we strive to respect your intents and wishes regarding your work, but receiving this license from you helps ensure that we are able to meaningfully use your work without needing to consider potential application risk.

— **4 Technical Standards & Stack** —

This section describes some practical details covering the technical areas we usually operate within. These are areas you may either already have experience with or may wish to develop into.

4.1 VRChat world logic

As the majority of our events take place within VRChat worlds, world logic is important for ensuring those events provide the hosts and attendees the functionality they need.

In general, most world logic is implemented in UdonSharp, as it makes complex behaviour easy to maintain and use across worlds. UdonSharp is a subset of C#, and so general C# experience will come in useful here. Experience with Unity behaviours and conventions is also necessary, but it is possible to learn these as they become relevant.

When implementing world logic, you should version control it and it is usually best practice to develop it to be agnostic to the world it is in. This allows it to be reused, reducing effort on your part, and allows you to break down the logic of a world into discrete maintainable parts.

It is also advised to format your code in a consistent manner, although we do not enforce any particular formatting scheme.

4.2 VRChat avatars

Avatars allow members of our community to express themselves, as well as offering functionality that can aid with the use of sign language.

No particular programming language knowledge is required for avatars. However, familiarity with Unity is necessary, and an understanding of software/computational concepts such as Finite State Machines is useful for working with animators, which is one of the primary and only methods of introducing logic to avatars.

4.3 Data recording and management

Helping Hands produces a lot of secondary format data that is used to facilitate our events. This includes:

- Lesson sheets, including auxillary data such as gesture annotation
- ShaderMotion video content
- Assets we produce for external parties, such as the event booths for PJKT and VRChat's NYE events.

Tracking and maintaining this data in a way that keeps it usable can be difficult. A lot of the time, these are best tracked with online documentation and file collation services, such as Google Drive, Google Docs, etc.

4.4 Community design

Helping Hands often needs graphics, models, and documentation to be created to communicate information about our events and values to the community. This includes artistic work as well as the standardization of design principles.

Usually we prefer to work in vector art when it is possible. This is because it makes it easier to revise, improve, and correct designs after the fact, and it makes it much easier to adapt work for multiple mediums (such as desktop, VR, print, and as software artifacts such as favicons).

Vector art in SVG format is possible to version control as text using Git, but raster art files like PNGs, or project files such as the ones produced by Inkscape, GIMP, Krita, Adobe Illustrator or Adobe Photoshop are considered 'binary' files. They can still be version controlled with Git as long as they are small enough, but if they are large and likely to change a lot, they may need to be version controlled separately (e.g. with manual backups to a Google Drive or so on).

Similarly, Blender files have a tendency to change a lot and grow in size, and so it is not always the best idea to version control them with Git. Blender has internal settings you can enable to automatically save several revisions of a file as you work on it, so these are highly recommended.

4.5 Web integrations development

This category includes services that interact with VRChat (such as the development of services providing JSON or images to be loaded dynamically into the game), as well as supporting services for other interfaces like Discord or the Helping Hands Website.

There is no particular fixed tech stack for these, but languages that make collaboration with programmers of varied skill levels are usually preferred, such as Python or TypeScript.

4.6 Helping Hands website

The website requires expertise in a variety of areas, including:

- User Interface and User Experience design
- Information hierarchy design
- Web programming, primarily TypeScript and React
- Graphical programming, mainly for displaying lesson or dictionary content
- Platform support and testing, as there are many browsers and devices the website can be accessed from.
- DevOps, E2E testing and deployment

It is most likely you can help in some capacity with at least one of these aspects. Simply using the website and providing useful feedback is always appreciated.

TypeScript is the most applicable programming language in this field, as it is what is used for the current website and is what is planned for future website iterations. It is like JavaScript but with type checking to make the behaviour and development process easier to predict.

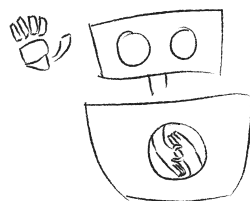
React is the component framework favored as well, primarily because it is easy to use and find resources for, however other component frameworks are a possibility in the future as well.

5 Conclusion and Handbook Revisions

This handbook is version controlled and the latest version is always accessible in the [Community Documentation repository](#). There are likely to be revisions as information gaps are discovered and the projects we work on and the processes we use mature.

If you have concerns or queries about the handbook content, or want to suggest changes or improvements, you can do so in the #dev-team-chat.

Thank you for your time in reading this handbook.



Bye bye for now!

A External links in document

- Helping Hands GitHub Organization

<https://github.com/HelpingHandsVR> 

- Git Version Control System

<https://git-scm.com/book/en/v2/Getting-Started-About-Version-Control> 

- Latest handbook revision

<https://github.com/HelpingHandsVR/community-documentation/blob/develop/staff/teams/development/handbook.pdf> 

B Bibliography

- [1] OWASP, "Cryptographic Storage · OWASP Cheat Sheet Series." [Online]. Available: https://cheatsheetseries.owasp.org/cheatsheets/Cryptographic_Storage_Cheat_Sheet.html 

Revision 15

849887a

2026-03-01T21:12:06Z

